

Background for Advanced Cryptography

2025

1 Introduction

This handout collects a number of fundamental facts that are used to reason / proof things in Cryptography. All these facts should be familiar to you, thus I only very briefly recap on them, and provides references to textbooks so you can read up further if necessary.

2 Number systems

Messages that we want to encrypt, and/or somehow manipulate, can be of different types, like sequences of words, numbers, pixels, etc. Eventually they all end up being represented as bit-strings, whereby the grouping of bits can vary.

It is important to be reasonably “fluent” in various representations. The most commonly used representations are: decimal, binary, and hexadecimal.

The decimal representation refers to representing numbers via the base (or radix) 10. Thus we have 10 digits (the numbers $0, 1, 2, \dots, 9$), and we use a positional number system to represent any number, whereby the east significant digit is the right-most digit. For instance, the number 19 is understood as $1 \cdot 10^1 + 9 \cdot 10^0$.

The binary representation refers to representing numbers via the base 2. Thus we have 2 digits (the numbers 0 and 1), and we use again a positional number system to represent any number. This implies that the least significant bit is the right-most in a sequence of bits. For instance, the decimal number 19 is represented as 10011 in binary which is understood as $1 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$.

The hexadecimal representation refers to representing numbers via the base 16. Thus we have 16 symbols $0, 1, 2, \dots, 9, a, b, \dots, f$. Thus the number decimal number 19 is represented as 13 in hexadecimal, which is understood as $1 \cdot 16^1 + 3 \cdot 16^0$.

Whereas it is often clear from the context what representation is used, in particular when

it comes to the binary representation, it is often helpful to indicate the representation to avoid confusion. Often this is done, but putting the basis as a subscript to the left of a number, e.g. 1_2 indicates that this is binary notation, and thus we have a number that equates to the decimal number 2. When it comes to indicating hexadecimal notation, then there exist a wide variety of conventions, like $0x3$, 3_{16} , 3_h , 3_{hex} .

A particular source of pain and grief when working with concrete examples in the context of computer systems is that when we have long messages, which are represented as a sequence of bytes (whatever representation is chosen for a byte), the the ordering of bytes can vary between processors, but also protocols. For instance, on a hardware level, most processor architectures store the least significant byte at the lowest memory address (of a sequence of addresses), which is called “little endianness”. The opposite ordering, i.e. the least significant byte is ordered “at the top” is then called big-endianness (this is used for instance in the Internet protocol suite). Thus when working with specifications of cryptosystems, or with programming languages, it is imperative to find out how they do the byte ordering.

3 Finite algebraic structures

Although typical messages are bit-strings, many cryptographic functions work over algebraic structures, such as finite groups, rings, and fields. The word “finite” implies that these algebraic structures have a finite number of elements. Typically this property is achieved by considering any number modulo “something”. The something can in principle be any positive integer. Special integers, namely those which are prime, are often beneficial.

An algebraic structure is a set of elements together with some operation(s); in other words, it is something that we can compute with (a set with no operations that can be done is not massively useful). The three main algebraic structures that you should know the definition of are the group, the ring, and the field.

A group is a set of elements G , together with an operator $\circ$, that satisfies four properties:

- (Closure:) If you take any two group elements and apply the operator, you get another group element: $\forall g, h \in G, g \circ h \in G$.
- (Identity exists:) There exists a unique element $e \in G$ such that for all elements $g \in G$, we have that $e \circ g = g = g \circ e$.
- (Inverses exist:) All group elements g have an inverse: $\forall g \in G : \exists h$ such that $g \circ h = e = h \circ g$.

- (Associativity holds): For all elements $g, h, i \in G$ the property holds that $g \circ (h \circ i) = (g \circ h) \circ i$.

If, in addition, also commutativity holds (i.e. $\forall g, h \in G : g \circ h = h \circ g$, then we call a group Abelian.

A ring is a set of elements G with two operators $\circ, \star$, whereby in relation to the operator $\circ$ all group laws are fulfilled, and in relation to the operator $\star$ the group laws without the existence of an inverse are fulfilled.

A field is a special type of ring where for both operators all group laws are fulfilled.

In classical cryptography, many cryptosystems are built on finite algebraic structures that are created by working modulo a prime. When working with prime moduli, we typically call them either p or q . A prime number has the property that it is only divisible by 1 and itself. When we consider numbers modulo a prime p , then it turns out that if we take either the addition or multiplication as an operation to work, then any result of $a + b$ or $a \cdot b \pmod{p}$ can be represented again as a value modulo p , and any number a also has an inverse modulo p . Thus the numbers modulo a prime p , plus an operation $+$ or $\cdot$ form a so-called group. If we take the numbers modulo a prime p and both operations, then we get a field.

3.1 Modular arithmetic

Modular arithmetic is fundamental to modern cryptosystems, and it is often the foundation of finite algebraic structures.

Given integers $a, b, N \in \mathbb{Z}$ we say that a is congruent to b modulo N

$$a \equiv b \pmod{N} \quad \text{iff} \quad N \text{ divides } b - a.$$

Often we are lazy and just write $a \equiv b$ if it is clear we are working modulo N .

Example: $16 \equiv 1 \pmod{5}$ since $16 - 1 = 3 \cdot 5$.

Be mindful that $\equiv$ and $=$ have a different mathematical meaning, however, different authors pay more or less attention to detail so sometimes these operators are used interchangeably (and thus wrongly).

For convenience we define the set

$$(\mathbb{Z}/N\mathbb{Z}) = \{0, \dots, N-1\},$$

which is the set of remainders modulo N . Note: some authors use $\mathbb{Z}_N = (\mathbb{Z}/N\mathbb{Z})$. It is clear that for any given N , any integer $a \in \mathbb{Z}$ is congruent modulo N to an element in the set $\mathbb{Z}_N$. This is because we can write

$$a = q \cdot N + r,$$

with $0 \leq r < N$ and clearly $a \equiv r \pmod{N}$.

The set $\mathbb{Z}_N$ has two operations defined on it.

- Addition

$$- (11 + 13) \pmod{16} \equiv 24 \pmod{16} \equiv 8 \pmod{16}.$$

- Multiplication

$$- (11 \times 13) \pmod{16} \equiv 143 \pmod{16} \equiv 15 \pmod{16}.$$

Facts: Given integers $a, b \in \mathbb{Z}$ we have

$$(a + b) \pmod{N} \equiv [(a \pmod{N}) + (b \pmod{N})] \pmod{N}$$

$$(a - b) \pmod{N} \equiv [(a \pmod{N}) - (b \pmod{N})] \pmod{N}$$

$$(a \cdot b) \pmod{N} \equiv [(a \pmod{N}) \cdot (b \pmod{N})] \pmod{N}$$

The multiplicative inverse of $a \in \mathbb{Z}_N$ is an element $b \in \mathbb{Z}_N$ with

$$a \cdot b \equiv b \cdot a \equiv 1.$$

We often write $b = a^{-1}$. Notice that because the inverse (if it exists) is in $\mathbb{Z}_N$, you must never get a fraction as a result when you do actual calculations.

Fact: $a \in \mathbb{Z}_N$ has a unique inverse modulo N iff a and N are relatively prime, i.e. $\gcd(a, N) = 1$.

Thus, if p is a prime then every non-zero element in $\mathbb{Z}/p\mathbb{Z}$ has an inverse, and consequently, we can use the set of non-zero elements to define a finite field (as mentioned before in these notes).

But when working with a composite modulus, there will be some elements that do not have an inverse.

3.1.1 Examples for composite modulus.

Consider $\mathbb{Z}_N$ with $N = 10$, then

- 3 has a multiplicative inverse, since $\gcd(3, 10) = 1$.
 - Indeed, $3 \cdot 7 \equiv 21 \equiv 1 \pmod{10}$.
- 5 has no multiplicative inverse, since $\gcd(5, 10) = 5$.
 - Indeed, we have the following table

$0 \cdot 5 \equiv 0 \pmod{10}$	$5 \cdot 5 \equiv 5 \pmod{10}$
$1 \cdot 5 \equiv 5 \pmod{10}$	$6 \cdot 5 \equiv 0 \pmod{10}$
$2 \cdot 5 \equiv 0 \pmod{10}$	$7 \cdot 5 \equiv 5 \pmod{10}$
$3 \cdot 5 \equiv 5 \pmod{10}$	$8 \cdot 5 \equiv 0 \pmod{10}$
$4 \cdot 5 \equiv 0 \pmod{10}$	$9 \cdot 5 \equiv 5 \pmod{10}$

3.1.2 First aid for computations over $\mathbb{Z}_{11}$.

There are a few times in the module where I give small examples for something by use of $\mathbb{Z}_{11}$. By $\mathbb{Z}_{11}$ we mean that we group all integers into residue classes modulo 11. We defined just before that $\mathbb{Z}_{11} = \mathbb{Z}/11\mathbb{Z} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$ and so any $a \in \mathbb{Z}$ is congruent to some $r \in \mathbb{Z}_{11}$ because we can write any $a = q \cdot 11 + r$ (and $0 \leq r < 11$).

Therefore, any $a \in \mathbb{Z}$ can be mapped to a residue in $\mathbb{Z}_{11}$ by performing a division with remainder. This can be done efficiently using a computer, and for small moduli, like 11, you can also do this with mental arithmetic. You can either “guess” q and then work out r , or if $a > 0$ is not too big than you can repeatedly subtract $N = 11$ from a to find r , or if $a < 0$ and not too big you can repeatedly add $N = 11$ to a in order to find r .

Here are some examples:

$13 = 1 \cdot 11 + 2 \equiv 2 \pmod{11}$	$-13 = -2 \cdot 11 + 9 \equiv 9$
$98 = 8 \cdot 11 + 10 \equiv 10 \pmod{11}$	$-1 = -1 \cdot 11 + 10 \equiv 10$
$127 = 11 \cdot 11 + 6 \equiv 6 \pmod{11}$	$-45 = -5 \cdot 11 + 10 \equiv 10$

Recall that for operations $+$, $-$, $\cdot$ and also $^{-1}$ we can apply the modulo reduction to any operator before and after the operation. This means that we can keep numbers small, which is often helpful when doing mental arithmetic. It is also imperative when working with very large moduli in the context of real world implementations.

Here are some more examples, including multiplications.

$$\begin{aligned}
3 + 12 \cdot 2 &\equiv 3 + 1 \cdot 2 = 5 \pmod{11} \\
34 - 45 \cdot 56 &\equiv 1 - 1 \cdot 1 = 0 \pmod{11} \\
-15 - 1 \cdot 3 &\equiv 7 + 10 \cdot 3 = 37 \equiv 4 \pmod{11} \\
-15 - 1 \cdot 3 &\equiv 7 - 3 = 4 \pmod{11}
\end{aligned}$$

Notice that there are multiple ways to achieve the same result. We have not done anything with modular inverses yet. For examples with small moduli, it can be helpful, if you do not want to use a calculator that does modular arithmetic, to just write down a modular inversion table. So here we go:

$1^{-1} \equiv 1 \pmod{11}$	because $1 \cdot 1 = 1 \pmod{11}$
$2^{-1} \equiv 6 \pmod{11}$	because $2 \cdot 6 \equiv 1 \pmod{11}$
$3^{-1} \equiv 4 \pmod{11}$	because $3 \cdot 4 \equiv 1 \pmod{11}$
$4^{-1} \equiv 3 \pmod{11}$	because $4 \cdot 3 \equiv 1 \pmod{11}$
$5^{-1} \equiv 9 \pmod{11}$	because $5 \cdot 9 \equiv 1 \pmod{11}$
$6^{-1} \equiv 2 \pmod{11}$	because $6 \cdot 2 \equiv 1 \pmod{11}$
$7^{-1} \equiv 8 \pmod{11}$	because $7 \cdot 8 \equiv 1 \pmod{11}$
$8^{-1} \equiv 7 \pmod{11}$	because $8 \cdot 7 \equiv 1 \pmod{11}$
$9^{-1} \equiv 5 \pmod{11}$	because $9 \cdot 5 \equiv 1 \pmod{11}$
$10^{-1} \equiv 10 \pmod{11}$	because $10 \cdot 10 \equiv 1 \pmod{11}$

Let us use this to do some more examples.

$$\begin{aligned}
2 - 9^{-1} &= 2 - 1 \cdot 9^{-1} \equiv 2 - 1 \cdot 5 = -3 \equiv 8 \pmod{11} \\
2 - 9^{-1} &= 2 - 1 \cdot 9^{-1} \equiv 2 + 10 \cdot 5 = 52 \equiv 8 \pmod{11} \\
-5^{-1} + 45 &\equiv 6^{-1} + 1 \equiv 2 + 1 = 3 \pmod{11}
\end{aligned}$$

3.2 Basic facts

The **order of an element** a of a group $(G, \cdot)$ is the smallest positive integer t such that $a^t = 1$.

The **order of a group/set** is the number of elements in that group/set.

An element a of a group $(G, \cdot)$ is a **generator** if the order of a is equal to the order of the group G .

Lagrange's Theorem

If G is a group of order (size) $n = \#G$ then for all $a \in G$ we have

$$a^n = 1.$$

Fact: the order t of an element $a \in G$ divides $n = \#G$.

So if $a \in (\mathbb{Z}/N\mathbb{Z})^*$ then $a^{\phi(N)} \equiv 1 \pmod{N}$, since $\#(\mathbb{Z}/N\mathbb{Z})^* = \phi(N)$.

Fermat's Little Theorem Suppose p is a prime and $a \in \mathbb{F}_p^*$ then

$$a^p \equiv a \pmod{p}.$$

Fermat's Little Theorem is a special case of Lagrange's Theorem.

4 Probability

A random variable is a function that maps elements from its domain (the sample space, representing the outcomes or events of an experiment) to some measurable space (often the real numbers). Nearly always, random variables are denoted by capital letters, and sometimes their realisations are denoted by lower case letters.

Example. The experiment could be a coin flip, resulting in the outcome that either Head or Tail lands on top:

- I call the function, aka random variable, that represents the coin flip C
- The sample space is the set of outcomes (head or tail): $\{H, T\}$
- I define C to map each outcome $\{H, T\}$ to a real value: $C : \{H, T\} \rightarrow \mathbb{R}$ such that

$$C(\omega) = \begin{cases} 1, & \text{if } \omega = \text{heads,} \\ -1, & \text{if } \omega = \text{tails.} \end{cases}$$

If a random variable X maps outcomes to some countable measurable space (i.e. one can associate each element with an element of the natural numbers), then X is called a **discrete** random variable. If X can take any value within an interval, it is called a **continuous** random variable. In our example, the random variable C is a discrete random variable, because the mapping goes to the two values $\{-1, 1\}$.

The idea that we model by the concept of a “random variable” is that the process that the function models is probabilistic. This means that if I repeat the experiment many

times, I might get different outcomes: if I perform a coin flip multiple times, some of the times I will see the head come up on top and sometimes the tail. Therefore it may be interesting to ask/know how likely it is for any outcome of the outcomes to occur.

If we know this likelihood then we call it either a **probability mass function, short pmf** (for discrete random variables) or we call it a **probability density function, short pdf** (for continuous random variables).

4.1 Discrete RVs

Example. Continuing with our example from before we could set out to define or determine the likelihoods for the outcomes Head and Tail. If we have built the coin, and we know that its weight is perfectly evenly distributed, then we might argue that the coin flip should result in a Head with the same likelihood than the Tail. So the probability mass function $f_C(c)$ would be

$$f_C(c) = \begin{cases} \frac{1}{2}, & \text{if } c = 1, \\ \frac{1}{2}, & \text{if } c = -1. \end{cases}$$

Another way of expressing the same thing is to write $\Pr[C = 1] = 1/2$ and $\Pr[C = -1] = 1/2$.

The probabilities of all disjoint outcomes sum up to 1, and the probability of any outcome must be a real number between 0 and 1. If we take one (set of) outcomes, and this occurs with probability p , then the complement (i.e. the probability that this event does not occur) must be $1 - p$.

In cryptography we are often interesting in how our knowledge of a random variables changes after we have seen another (perhaps related) random variable. For instance, we know that some unknown plaintext m was drawn from some distribution M with known pmf. Then this plaintext gets encrypted under some unknown secret key, leading to some ciphertext c . We might want to know what is the probability of the plaintext m given that we known now c ? To answer such a question, we need to know something about the joint distribution of plaintexts and ciphertexts. A joint distribution is the probability distribution of all possible pairs of outcomes. If the outcomes from the distributions are sampled independently, then the joint distribution is simply the product of the distributions.

Example. Let's switch from coin flips to randomly sampled bits. Suppose we have an encryption function that take a bit as input, and samples a bit at random from a uniform distribution independently of the input. Then encryption function exclusive-ors the bits to produce a ciphertext: $c = m \oplus k$. This implies we have the following key and message

distributions:

$$f_M(m) = \begin{cases} p, & \text{if } m = 0, \\ 1 - p, & \text{if } m = 1, \end{cases}$$

and

$$f_K(k) = \begin{cases} \frac{1}{2}, & \text{if } k = 0, \\ \frac{1}{2}, & \text{if } k = 1. \end{cases}$$

We can now produce the joint distribution of M and K , by looking at the probabilities of all pairs of outcomes:

	$\Pr[m = 0] = p$	$\Pr[m = 1] = 1 - p$
$\Pr[k = 0] = 1/2$	$\Pr[m = 0 \wedge k = 0]$	$\Pr[m = 1 \wedge k = 0]$
$\Pr[k = 1] = 1/2$	$\Pr[m = 0 \wedge k = 1]$	$\Pr[m = 1 \wedge k = 1]$

Plugging in what we know from the given pmfs, and utilising that the key was sampled independently from the input we get the joint distribution of M and K :

	$\Pr[m = 0] = p$	$\Pr[m = 1] = 1 - p$
$\Pr[k = 0] = 1/2$	$\Pr[m = 0 \wedge k = 0] = p \cdot \frac{1}{2} = \frac{p}{2}$	$\Pr[m = 1 \wedge k = 0] = (1 - p) \cdot \frac{1}{2} = \frac{1-p}{2}$
$\Pr[k = 1] = 1/2$	$\Pr[m = 0 \wedge k = 1] = \frac{p}{2}$	$\Pr[m = 1 \wedge k = 1] = \frac{1-p}{2}$

Closely related to joint distributions are conditional distributions. A conditional distribution is closely related to the joint distribution:

$$\Pr[M = m | C = c] = \frac{\Pr[M = m \wedge C = c]}{\Pr[C = c]}. \quad (1)$$

An incredibly useful tool is Bayes' theorem which allows us to "swap" the random variables in a conditional probability:

$$\Pr[M = m | C = c] = \frac{\Pr[C = c | M = m] \cdot \Pr[M = m]}{\Pr[C = c]}. \quad (2)$$

4.2 Continuous RVs

A continuous random variable is defined over a space that is no longer countable. Therefore a continuous random variable almost never takes any exact value, i.e. it makes no sense to look at $\Pr[X = x]$. Instead one typically looks at situations like $\Pr[X \leq x]$ or

$\Pr[x_1 \leq X \leq x_2]$, and defines them via integrating over an interval (e.g. $[0, x]$ or $[x_1, x_2]$) using the given probability density function.

In the context of side-channel attacks components that make up the power consumption at time point t are the sum of contributions of many gates and interconnects. The sum of these individual contributions is best understood as a continuous random variable. In particular, it is commonly accepted to model the contribution of the noise components via a continuous Gaussian probability distribution¹.

The probability density function describing the Gaussian (aka “normal”) distribution (3) $\mathcal{N}(\mu, \sigma^2)$ only depends on the parameters μ (expectation) and σ (standard deviation, square root of variance), where $-\infty < \mu < \infty$ and $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2 \cdot \pi} \cdot \sigma} \cdot \exp \left(-\frac{1}{2} \cdot \left(\frac{x - \mu}{\sigma} \right)^2 \right) \quad (3)$$

If we have $\mu = 0$ and $\sigma = 1$ then we call this a **standard** normal distribution and give it the name ϕ . Evidently, any normal distribution is related to the standard normal distribution via $f(x) = \frac{1}{\sigma} \phi \left(\frac{x - \mu}{\sigma} \right)$.

Equivalently, if Z is standard normal, then $X = \sigma Z + \mu$ is normal. And conversely, if X is normal with parameters (μ, σ) then $Z = (X - \mu)/\sigma$ is normal.

The probability that a standard normal distributed variable X takes an outcome smaller or equal to x is then given as:

$$\Pr[X \leq x] = \phi(x) = \frac{1}{\sqrt{2 \cdot \pi}} \int_{-\infty}^x e^{-t^2/2} dt.$$

Statements that we made before in the context of discrete random variables, regarding conditional and joint distributions can be also made when working with continuous random variables. However, conditioning on something like $Y = y$ when Y is continuous does not make sense. One can define “alike” things for continuous random variables, however, in the context of side channels we will work with a discrete random variable and a continuous random variable, and we will make sure to condition on the discrete random variable.

4.3 Expectation, Variance, and Covariance

Consider again the situation where we repeatedly conduct an experiment and look at the outcomes. We now assume that each repetition happens in the same way and inde-

¹One can now argue that trough the process of measurement via a **digital** oscilloscope the resulting traces should be understood as tuples of discrete random variables, but many trace processing techniques that are applied in practice (filtering, smoothing), may well lead to trace points that are still best described as continuous random variables.

pendently of previous experiments, and experiments that we do thereafter. Sometimes questions of what the kind “what is the average outcome” and “how much does the average vary” are interesting in practice.

The **expectation of a discrete random variable** X is defined as

$$E[X] = \sum_i x_i \cdot f_X(x)$$

, whereby the sum runs over all outcomes of X , and $f_X(x)$ is the probability mass function of X .

The **expectation of a continuous random variable** X is defined as

$$E[X] = \int_{-\inf}^{\inf} x \cdot f(x) dx$$

, whereby $f(x)$ is the probability density function of X .

This definition takes into account that some outcomes might occur more often than others (because they are more likely). If all outcomes are all equally likely, then the expectation equals the arithmetic average.

It is useful to know that, because of its definition, the expectation satisfies a number of linearity properties:

$$\begin{aligned} E[X + Y] &= E[X] + E[Y] \\ E\left[\sum_{i=1}^N a_i \cdot X_i\right] &= \sum_{i=1}^N a_i \cdot E[X_i] \\ E[a] &= a \end{aligned}$$

The **variance of a random variable** X is defined as the squared deviation from the expectation of X :

$$Var(X) = E[(X - E(X))^2].$$

This definition applies to both discrete and continuous random variables.

One can expand out the above definition for the variance and derive an alternative form:

$$\begin{aligned} Var(X) &= E[(X - E[X])^2] = E[(X^2 - 2XE[X] + E[X]^2)] \\ &= E[X^2] - 2E[X]E[E[X]] + E[X]^2 \\ &= E[X^2] - E[X]^2 \end{aligned}$$

This expression is sometimes more convenient to work with from a mathematical point of view. However, from a practical aspect of implementing this equation, caution is required. This is because the terms in the last line can be very similar and therefore they may simply cancel out when naively implemented.

Because the variance is defined as squared difference, it must be positive. The variance of a constant is zero. The variance has some useful properties that inherits because of the linearity of the expectation:

$$\begin{aligned} \text{Var}(X + a) &= \text{Var}(X) \\ \text{Var}(aX) &= a^2 \text{Var}(X) \\ \text{Var}(aX \pm bY) &= a^2 \text{Var}(X) + b^2 \text{Var}(Y) \pm 2ab \text{Cov}(X, Y). \end{aligned}$$

The expression $\text{Cov}(X, Y)$ refers to the **covariance between X and Y** . The covariance measures the joint variability of X, Y . It is defined as

$$\text{Cov}(X, Y) = E[(X - E[X]) \cdot (Y - E[Y])].$$

Using the linearity properties of the expectation, one can rewrite this expression as:

$$\begin{aligned} \text{Cov}(X, Y) &= E[(X - E[X]) \cdot (Y - E[Y])] \\ &= E[XY - XE[Y] - E[X]Y + E[X]E[Y]] \\ &= E[XY] - E[X]E[Y]. \end{aligned}$$

This latter expression makes the relationship of the covariance to the joint distribution of X, Y perhaps more apparent, but like in the alternative form of the variance you will notice that a difference has to be computed, which can be problematic when the two values are very close. Wikipedia provides an excellent overview for techniques that enable accurate variance calculations and estimations: https://en.wikipedia.org/wiki/Algorithms_for_calculating_variance.

Related to the covariance is the concept of correlation: it is a measure of how strongly two random variables are related to one another. It is important to bear in mind that relatedness does not imply an “cause and effect” relationship, or, if there is a causal relationship then one has to be careful how this looks like.

There are multiple correlation measures defined in statistics. A very popular one is called Pearson’s product moment coefficient. It is essentially a normalised version of the covariance:

$$\rho(X, Y) = \frac{\text{Cov}(X, Y)}{\text{Var}(X) \cdot \text{Var}(Y)}.$$

If two random variables are independent, then their covariance, and thus the correlation between them equals zero. It is not true that if one observes zero correlation, then X and Y are necessarily independent.

5 (Perfectly) Secret Encryption

A symmetric encryption $\mathcal{E}$ scheme is given by three algorithms: Gen , Enc , and Dec .

- $Gen : \mathcal{K} \xrightarrow{\$} \textcolor{red}{sk}$ generates a secret key via randomly sampling an element from the keyspace
- $Enc : \mathcal{M} \times \mathcal{K} \rightarrow \mathcal{C}$ maps an element from the plaintext space to the ciphertext space (using an element from the key space)
- $Dec : \mathcal{C} \times \mathcal{K} \rightarrow \mathcal{M}$ recovers the plaintext from the ciphertext (using the secret key)

A symmetric encryption scheme is said to be correct if:

$$\forall m \in \mathcal{M} \text{ and } \forall \textcolor{red}{sk} \in \mathcal{K} : Dec_{\textcolor{red}{sk}}(Enc_{\textcolor{red}{sk}}(m)) = m.$$

A common assumption (dating back to the Dutch cryptographer Kerckhoffs) is that except for the (shared) **secret** key $\textcolor{red}{sk}$, all algorithms and parameters of an encryption scheme are public knowledge.

Adversary's powers and goals need to be considered and clearly stated when reasoning about the security and usefulness of any given system.

Powers:

- Adversaries with unlimited computational power: security in this context is called 'information theoretic security' or 'unconditional security' or 'perfect secrecy'.
- Adversaries with finite computing power: computational security.
- Adversaries who have access to a physical implementation of the system, which may be observed and/or manipulated (physical security; leakage resilience).

Goals:

- Key recovery: once an adversary has the secret key they can read all messages that were ever encrypted with that key.
- Plaintext recovery: perhaps the adversary has observed ciphertexts and wants to recover the associated plaintexts.

If key recovery is possible, then this implies a full break of a symmetric encryption system as it also implies plaintext recovery. The ability to get some plaintext may not imply the ability to recover the key. Many practical attacks thus target key recovery (e.g. linear and differential cryptanalysis, side channel and fault attacks) and they target structural problems in concrete symmetric encryption schemes.

5.1 Perfect secrecy by Shannon

The ultimate goal of any symmetric encryption scheme though is that to protect some message. Consequently the typical way of defining security for symmetric encryption is to formalise the idea that an adversary “does not learn anything about a message upon seeing its ciphertext”. This idea dates back to Shannon who also formalised it. He then showed that for the so-called “one-time pad encryption” the property holds even for an adversary who has unlimited computational power. He called this property “perfect secrecy” (the perfect refers to the unlimited power). In modern cryptography we use the same idea, but without the “perfect”, i.e. we construct schemes for which we can prove the property for only computationally bounded adversaries.

Definition. *An encryption scheme Gen, Enc, Dec with message space $\mathcal{M}$ is **perfectly secret** if for every probability distribution over $\mathcal{M}$, every message $m \in \mathcal{M}$, and every ciphertext $c \in \mathcal{C}$ for which $\Pr[C = c] > 0$:*

$$\Pr[M = m | C = c] = \Pr[M = m].$$

Remember that except for the key sk which is used to transform a message into a ciphertext, all other parameters/inputs/functions are public knowledge. Thus, the distribution of the message space is also public knowledge. For instance, if the adversary knows that the message is some Italian text, then they know that some words are more likely to occur than others. So we allow an adversary to know something about the message!

This definition should be decoded as follows: the left hand of the equation represents what the attacker knows after seeing a ciphertext; the right hand side represents what the attacker knows without seeing the ciphertext.

Thus: even after the adversary sees some ciphertext, their knowledge about the (distribution of the) plaintext is the same as before. The ciphertext thus gives them no further information about the plaintext.

Shannon was able to prove this property for the one-time pad, but it is a bit unwieldy to work with at times.

5.1.1 One-time pad

The encryption scheme OTP has a message space $\mathcal{M}$, a ciphertext space $\mathcal{C}$ and a key space $\mathcal{K}$ with $|\mathcal{K}| = |\mathcal{C}| = |\mathcal{M}| = 0, 1^n$. *Gen* samples k uniformly at random from $\mathcal{K}$ for a one time use. We define $c = \text{Enc}_k(m) = m + k \bmod 2$ (i.e. bit-wise XOR), and decryption likewise to be $m = \text{Dec}_k(c) = c + k \bmod 2$.

Thus whenever we use the OTP to encrypt a message, a new key must be sampled randomly from the key space. This key must be of equal length to the message.

Note that the addition modulo 2 corresponds to a binary exclusive-or operation. This is why you will often find encryption and decryption described as $c = m \oplus k$ and $m = c \oplus k$. To emphasise that each k may only be used once between two parties, we often include key generation in the encryption function.

Why is this simple cryptosystem perfectly secure? Consider the properties of the addition modulo 2, and fix a pair of message and ciphertext (m, c) . Given the encryption function, we have that $c = m \oplus k$, which implies that $k = m \oplus c$. Now let's consider the definition for perfect secrecy: $\Pr[M = m | C = c] = \Pr[M = m]$, no matter which m, c pair we consider.

Let us show, by using the definition of the OTP encryption/decryption, and the fact that message, ciphertext and keyspace are all of equal size, that the OTP satisfies the property of perfect secrecy.

$$\begin{aligned}
\Pr[M = m|C = c] &= \frac{\Pr[C = c|M = m] \cdot \Pr[M = m]}{\Pr[C = c]} \\
&= \frac{\Pr[c = \text{Enc}_K(M)|M = m] \cdot \Pr[M = m]}{\Pr[C = c]} \\
&= \frac{\Pr[c = K \oplus m] \cdot \Pr[M = m]}{\Pr[C = c]} \\
&= \frac{2^{-n} \cdot \Pr[M = m]}{\Pr[C = c]} \\
&= \frac{2^{-n} \cdot \Pr[M = m]}{\sum_m \Pr[C = c|M = m] \cdot \Pr[M = m]} \\
&= \frac{2^{-n} \cdot \Pr[M = m]}{\sum_m 2^{-n} \cdot \Pr[M = m]} \\
&= \frac{2^{-n} \cdot \Pr[M = m]}{2^{-n}} \\
&= \Pr[M = m]
\end{aligned}$$

As you can see the argument here heavily relies on the property of the exclusive-or that is used for encryption. One cannot make the same argument if we, e.g., consider a modern block cipher like AES, even if we restricted the use to a single block encryption.

5.2 Indistinguishable encryptions

One of the major intellectual advances in modern cryptography was to provide an alternative definition for “secrecy”, which works as follows.

We play a game with an adversary. In this game, we allow the adversary to choose two message m_0, m_1 of equal length. One of the two messages gets encrypted and its encryption is passed back to the adversary who needs to tell if this is the encryption of m_0 or if it is the encryption of m_1 . If they can determine correctly which of the two messages they were given (i.e. if they can win the game), then there must be some information in the ciphertext. Thus we say the scheme is insecure. Otherwise, if they cannot win the game, we say that the scheme is secure. This game is depicted in Figure 1.

Bellare and Goldwasser showed that this game is mathematically equivalent to Shannon’s definition when giving the adversary infinite computing power. The notion, without assuming an infinitely powerful adversary, is also often called “semantic security”, and, “indistinguishability of encryptions”. The latter name explains one part of the abbreviation given for the game in the Figure caption. The caption reads “Exp:ind-pas”: “Exp”

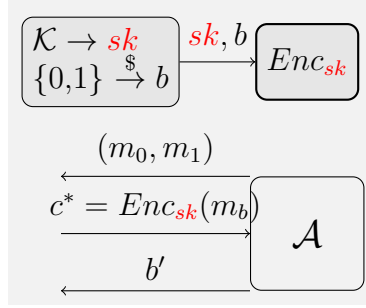


Figure 1: Exp ind-pas

stands for experiment; “ind” stands for indistinguishability of ciphertexts, and “pas” stands for “passive”. A passive adversary is one that has no help at all in their task.

In modern cryptosystems we tend to assume that an adversary has some help. In the case of symmetric encryption, we assume that for some “period of time”, the adversary can interact with an encryption oracle. This oracle uses the same secret key that is used to produce the “challenge ciphertext” c^* . Thus we give the adversary to chance to “learn something if there is something to be learned” as part of the game. We show this version of the game in Figure 2.

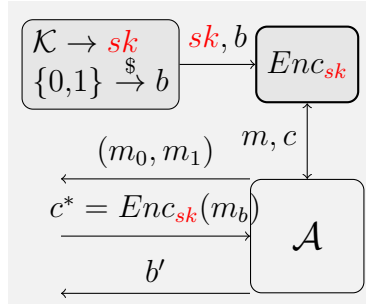


Figure 2: Exp ind-cpa

Clearly the “amount of time” (or equivalently the “amount of plaintext-ciphertext pairs”) that they are allowed to get is important here. If they could get all possible pairs, thus if they can search the entire input space that the encryption scheme supports, then obviously at least one of the pairs would match either (m_0, c^*) or (m_1, c^*) and they win the game. Doing so is obviously equivalent to the effort of a brute-force key search. Thus, we must ensure in practice that input, key, and ciphertext space are “large enough” to make any such brute-force search approach infeasible in practice. This is why we define modern encryption schemes over very large block and key sizes (at least 128 bit).

Note. Even more is required for practical symmetric cryptography to satisfy the notion of indistinguishability. Consider for instance the following adversarial strategy to choose the challenge messages: $m_0 = \underbrace{0 \dots 0}_n || \underbrace{0 \dots 0}_n$ and $m_1 = \underbrace{0 \dots 0}_n || \underbrace{1 \dots 1}_n$, when considering a modern encryption scheme like AES. The two texts consist of two blocks of either all 0 bits or all 1 bits. Suppose that they ensure that each block is exactly the number of bits that AES encryption can take as input. Because each message is twice the size of what AES can encrypt, the encryption function must be applied to each block independently. If ECB mode is chosen, then $\underbrace{0 \dots 0}_n$ encrypts to some ciphertext c_0 and $\underbrace{1 \dots 1}_n$ encrypts to some ciphertext c_1 . Consequently, m_0 encrypts to $c_0 || c_0$ and m_1 encrypts to $c_0 | c_1$. Thus an adversary can win this game easily (even without any help from an encryption oracle) and this explains why we must always use symmetric encryption schemes in a mode of operation that adds randomness somehow to “diversify” encryptions.

6 AES

AES encrypts data of a fixed block length (128 bits) under a key, which can either have 128, 192, or 256 bits. It is common practice to use the abbreviation AES-128, or simply AES, in order to refer to AES with 128-bit keys. There are few differences between AES-128, AES-192 and AES-256: the number of rounds increases and the key schedule changes accordingly. We focus on AES-128 (aka AES) only: to extend what we describe in this handout to AES with longer keys, one has to adapt attacks to cope with the fact that only 128 bits of those longer keys leak in a single round.

6.1 Structure of the AES Encryption

The following, brief, description is specific to AES-128 (aka AES with a 128 bit key). In AES the input data (128 bits, aka 16 bytes) and the key are represented as an array of bytes with four rows and four columns, see Figure 3. The data array is typically called the *state*. AES operates as a typical iterated cipher: a round transformation (in the form of an SP network) is repeatedly applied to the state. The round keys are also computed in an iterative fashion, and the round key data is analogously arranged to the state. It is important to notice how the elements of the state are indexed: we run through *column*-wise. Decryption works similarly, and there are different ways to implement it (e.g. via the equivalent inverse cipher). If any of these statements do not sound familiar to you, it would be best to revise the notes about AES implementations that are part of this unit.

There are a number of implementation strategies to map the high-level description A to some concrete implementation goal. In software, in the absence of a dedicated instruction set for AES operations, one has the choice between an 8-bit implementation (with

m_0	m_4	m_8	m_{12}
m_1	m_5	m_9	m_{13}
m_2	m_6	m_{10}	m_{14}
m_3	m_7	m_{11}	m_{15}

k_0	k_4	k_8	k_{12}
k_1	k_5	k_9	k_{13}
k_2	k_6	k_{10}	k_{14}
k_3	k_7	k_{11}	k_{15}

Figure 3: State and Key Layout (numbering convention) for AES

reasonable memory footprint), or a high-throughput 32-bit implementation (e.g. a so-called T-table implementation option that comes with a considerable memory footprint). We will focus our attention on an 8-bit implementation (i.e. no T-tables), although it is perfectly feasible (and sensible) to utilise 32-bit instruction for some components of the round.

Alg. A: AES Encryption

Provided an input m and key k , this delivers an AES encryption c of the input as a result.

```

input: m, k
output: c
state = m
( $rk_1, rk_2, \dots, rk_{10}$ ) = KeyExpansion(k)
AddRoundKey(state, k)
for r = 1 step 1 to 9
    SubBytes(state)
    ShiftRows(state)
    MixColumns(state)
    AddRoundKey(state,  $rk_r$ )
end
SubBytes(state)
ShiftRows(state)
AddRoundKey(state,  $rk_{10}$ )
c = state;

```

7 Statistics

Statistics is concerned with questions around estimation, quantification, and comparison of distributions (and/or their key parameters) when these distributions are unknown and

must be estimated from data.

We have just such a situation when we want to understand how well side channel attacks work: we don't really know the distribution of leakage traces (and/or trace points), but we can sample from the distribution. The question that we then have, in the context of attacks is, how well do our model and key dependent predicts match our observations from the real device.

An incredibly important concept in this regard is that of the “sampling distribution” of a distributional parameter such as the mean, variance. Also the correlation, which is often used in attacks has such a sampling distribution. Together with another salient concept (hypothesis testing), it is the main vehicle to work out how many traces one should acquire. It is also useful to work out how characteristics of side channels, such as the signal to noise ratio, impact on how many traces will be needed.

7.1 Sampling distribution of the correlation estimate r

The concept of a sampling distribution is helpful in understanding the next ideas that enable us to provide a rule of thumb for the number of needed power traces. To introduce this concept I suggest a thought experiment. Suppose that we would repeat an experiment that samples D traces from a device (with a fixed key k^*) and calculates a DPA attack with some known inputs, i.e. in our thought experiment we produce several distinguishing vectors $\mathbf{r}$. Let's just focus on the estimates for the correct key and correct time. These individual correlation coefficients are unlikely to be exactly identical: we would expect them to be very similar, but because traces are noisy, we wouldn't expect them to be the same. Hence the estimates r are in fact realisations of a random variable. They have a distribution, which is called the sampling distribution.

In the case of correlation turns out to be quite involved and tricky to work with. However, if a sufficiently large number of traces (≥ 30) can be measured, and the (true) correlation is small (i.e. ≤ 0.2) a transformation that is due to Fisher can be used to map the random variable R to a random variable Z_1 that has a normal distribution, see (4). The mean of Z_1 is then given by μ , see (5), and the variance is σ^2 , see (6). The value n denotes the number of samples (aka number of traces D). Note that the fraction $\rho/(2 \cdot (n - 1))$ approaches zero for large n and can thus be omitted from the formula. Observe that the variance indicates that our estimate gets better the more traces we have (more traces means smaller variance).

$$Z_1 = \frac{1}{2} \ln \frac{1+R}{1-R} \tag{4}$$

$$\mu = \frac{1}{2} \ln \frac{1+\rho}{1-\rho} + \frac{\rho}{2 \cdot (n-1)} \tag{5}$$

$$\sigma^2 = \frac{1}{n-3} \tag{6}$$

Now that we have reasoned that the any estimate r that we compute is a realisation of the sampling distribution of R , we turn our attention back to the question of how many traces we need such that we can distinguish a non-zero r from 0 with a high probability. I describe the process in the subsequent section: **this is for information only, and I will not examine any maths associated with underpinning statistical machinery.**

7.2 Hypothesis Tests

In statistics, we use *hypothesis test* and their associated *confidence intervals* to test how close random variables are based on sample statistics. In a hypothesis test, we test whether a certain hypothesis that we make is true or not. For instance, we can test whether R is equal to R_0 or not. In our use case, we formulate two hypotheses. The first (null) hypothesis is that $H_0: R = R_0$. The second, alternative hypothesis is $H_1: R \neq R_0$.

Confidence intervals can be easily constructed for standard normal variables. Therefore we use Fisher's transformation to map R to Z_1 and then normalise Z_1 to get $Z = (Z_1 - \mu)/\sigma \sim \mathcal{N}(0, 1)$. We do the same of to map 0 correlation to a normal distributed variable Z_0 .

$$\begin{aligned}
R_0 &\mapsto Z_0 = \frac{1}{2} \cdot \ln \frac{1 + R_0}{1 - R_0} \\
\mu_0 &= E(Z_0) = \frac{1}{2} \cdot \ln \frac{1 + \rho_0}{1 - \rho_0} + \frac{\rho_0}{2 \cdot (n - 1)} \\
\sigma^2 &= Var(Z_0) = \frac{1}{n - 3} \\
R_1 &\mapsto Z_1 = \frac{1}{2} \cdot \ln \frac{1 + R_1}{1 - R_1} \\
\mu_1 &= E(Z_1) = \frac{1}{2} \cdot \ln \frac{1 + \rho_1}{1 - \rho_1} + \frac{\rho_1}{2 \cdot (n - 1)} \\
\sigma^2 &= Var(Z_1) = \frac{1}{n - 3}
\end{aligned}$$

The distribution of the difference $Z_0 - Z_1$ is a normal distribution $Z_0 - Z_1 \sim \mathcal{N}(\mu_0 - \mu_1, \sqrt{2} \cdot \sigma)$. Hence, $((Z_0 - Z_1) - (\mu_0 - \mu_1))/(\sqrt{2} \cdot \sigma)$ has a standard normal distribution. The two-sided confidence interval for $\mu_0 - \mu_1$ is defined as:

$$\left[(Z_0 - Z_1) - z_{1-\alpha/2} \cdot \sqrt{\frac{2}{n-3}}, (Z_0 - Z_1) + z_{1-\alpha/2} \cdot \sqrt{\frac{2}{n-3}} \right]. \quad (7)$$

The term $z_{1-\alpha/2}$ relates to the probability α that the interval will actually contain our sample estimate: we want this to be large so we adjust the parameter $1 - \alpha/2$ to be 0.9999, which means that $z_{0.9999} = 3.719$.